PROJECT DOCUMENTATION

Project Management System (PMS)

Software Engineering Mini Project

Team 11

Document field	Details
Purpose	Consolidated project reference and Sprint 1 documentation record
Version	1.0 — Draft for team review
Technology baseline	Python, Django, Django Templates, HTML/CSS/JavaScript
Database baseline	PostgreSQL in architecture/schema documents; verify actual configuration

1. Purpose and Use

This document provides a single navigation and coordination reference for Team 11's Project Management System (PMS). It summarizes the project purpose, team responsibilities, requirements baseline, technology and architecture, repository conventions, database design, testing approach, Sprint 1 plan, documentation controls, and known items requiring confirmation. It complements rather than replaces the Software Requirements Specification (SRS), Software Architecture and Design Specification (SASD), architecture design, database schema design, and Software Test Plan (STP).

The SRS describes intended product scope; architecture and schema documents describe proposed designs; the STP describes the testing strategy. These documents alone do not prove that a feature has been implemented or passed testing.

2. Project Overview

The PMS is planned as a centralized web application for teams to plan, organize, execute, monitor and report project work. Intended capabilities include authentication and role-based access, project/task management, Kanban/workflow management, estimation, sprint/milestone planning, dashboards, notifications, collaboration, search/filtering, reporting and activity logging.

The SRS identifies three user classes: Admin, Project Manager and Team Member. The permissions and exact scope of each role should follow the approved requirements and be enforced by the application.

2.1 Objectives

• Provide a shared location for project and task information.
• Support planning, assignment, status tracking and collaboration.
• Give project managers visibility into progress and reports.
• Keep implementation modular, testable and maintainable.
• Maintain traceability between requirements, implementation tasks, tests and documentation.

3. Team and Responsibilities

The general responsibilities below follow the supplied README. Sprint task assignments are listed separately because Jira assigns specific Sprint 1 work items.

Member	Sprint 1 Jira assignment shown
Shivansh Sharma	SCRUM-6 Repository; SCRUM-11 CI/CD
Shibi R Shetty	SCRUM-4 Architecture; SCRUM-5 Database; SCRUM-7 Project management; SCRUM-9 Backend foundation
Sheetal S Bevinakatti	SCRUM-8 Frontend foundation; SCRUM-10 Unit/integration tests
Shivanagouda I Patil	SCRUM-3 Requirements analysis; SCRUM-12 Documentation

4. Requirements Baseline

The SRS v1.0 is the primary functional and non-functional requirements reference. The SCRUM-3 analysis inventories the FR/NFR set and proposes a Sprint 1 scope. The full SRS must not be interpreted as a commitment to implement every feature during Sprint 1.

Requirement group	Intended capability	Sprint 1 relevance
FR1 — Authentication	Registration, login/logout, password reset, role-based access	Foundation relevance; no dedicated authentication implementation item is shown in the current backlog.
FR2 — Project management	Create/manage projects and team membership	Direct functional focus through SCRUM-7; exact subset needs team confirmation.
FR3 — Task/issue management	Create, assign and update tasks/issues	Broader product scope; no separate implementation item is shown in the Sprint 1 backlog.
FR4–FR5 — Kanban/workflow	Board status and allowed workflow transitions	Broader product scope; Sprint 1 implementation depth not confirmed.
FR6–FR7 — Estimation/sprints	Effort estimates and sprint/milestone planning	Broader product scope; later work unless explicitly included.
FR8–FR13 — Visibility/collaboration	Dashboards, notifications, comments, search, reports, activity logs	Broader product scope; not individually assigned as implementation tasks in the shown Sprint 1 backlog.

Requirement group	Intended capability	Sprint 1 relevance
NFRs	Performance, scalability, security, usability, reliability, maintainability, portability, backup/recovery, interoperability	Consider relevant qualities during foundation work; do not claim system-wide compliance without evidence.

5. Technology and System Architecture

Layer / component	Responsibility
Presentation	Browser interface using Django Templates, HTML, CSS and JavaScript.
Django web layer	URL routing, views, forms, sessions/authentication, request handling and validation.
Feature modules	Accounts/access, projects, tasks/issues, Kanban/workflow, estimation/sprints, dashboards/search/reporting, collaboration/notifications/activity.
Shared business rules	Role/permission checks, required-field validation, workflow transition rules and consistent error handling.
Data access	Django ORM, models, migrations and transactions.
Database	PostgreSQL is the architecture/schema baseline; verify actual configured database.
Version control/automation	Git/GitHub for source control; GitHub Actions planned for automated checks and tests.

The supplied architecture design describes a layered modular monolith: one Django application organized into logical feature modules, not independent deployed microservices.

5.1 Typical request flow

A user acts in the browser; the request reaches Django; authentication, authorization and input checks are applied; the relevant feature module executes business rules; the ORM reads or writes the database; and Django returns a page or appropriate error response.

5.2 Architecture references

Use **PMS_System_Architecture_Design.pdf** for the high-level architecture diagram and request flow. Use **SASD_Specification.docx** for the fuller design specification, component descriptions, security considerations and interfaces. Keep the two aligned as decisions are finalized.

6. Database Design

The supplied **pms_database_schema_design.pdf** is labelled a proposed logical design for team review. It describes a PostgreSQL relational schema and reference DDL for users, projects, project membership, tasks, workflows/states/transitions, sprints, milestones, comments, attachment metadata, notifications and activity logs.

The schema document explicitly states that it is a proposed design, not evidence that the database has been created or tested. Django models and migrations should become the implementation source of truth after the team confirms the database choice and business rules.

6.1 Database artifacts

• **pms_database_er_diagram.png** — visual entity/relationship representation.
• **pms_database_schema_design.pdf** — logical schema, entity definitions and constraints.
• **pms_database_schema_postgresql.sql** — PostgreSQL reference DDL; verify against actual Django models/migrations before treating it as deployment truth.

6.2 Decisions requiring confirmation

• Confirm PostgreSQL as the shared development database. The README mentions PostgreSQL in production and SQLite for local development/testing, while the architecture and schema focus on PostgreSQL.
• Confirm schema fields, role representation and constraints map to Django's authentication model and migrations.
• Record approved changes in the schema document and repository.

7. Repository and Setup Reference

The supplied README identifies the repository as Project-Management-System-SE and documents a Django backend, templates, static assets, documentation and a GitHub Actions workflow

Path	Purpose
README.md	Project overview, architecture, team guide and setup instructions.
backend/manage.py	Django administrative utility.
backend/requirements.txt	Python dependencies.
backend/config/	Django settings and main URL configuration.
backend/accounts/	Authentication and profile module.
backend/projects/	Project management module.
backend/tasks/	Task and Kanban management module.
backend/tests/	Unit/integration test suite, if organized here.
templates/ and static/	Django HTML templates and static assets.
docs/	SRS, test plan, architecture, requirements matrix, proposal, SCRUM-3 analysis, SASD and database artifacts.
.github/workflows/tests.yml	Planned GitHub Actions workflow for automated checks and tests.

8. Testing and Quality Assurance

8.1 SCRUM-10 test scope

The supplied test files are backend/tests/test_forms.py, backend/tests/test_models.py and backend/tests/test_views.py. The visible code covers:

Test file	Coverage visible in supplied code
test_forms.py	Accepts valid project data; requires project name; rejects status values outside documented choices.
test_models.py	Password hashing and email authentication identifier; superuser flags; default project status; due-date constraint; unique project membership.
test_views.py	Dashboard context and alias; project list/form context; authenticated project creation and ownership; anonymous creation rejection; invalid project data rejection.

These examples reflect the supplied files and do not prove every SRS requirement has been tested. The STP also lists Selenium for UI automation, Postman for API testing, JMeter for performance testing, and Jira/GitHub Issues for defect tracking; the supplied evidence does not show that all these tools have been configured or used.

8.2 Test execution results

Metric / check	Latest reported result	Evidence / note
Total tests executed	59	Latest terminal screenshot and teammate message
Passed	59	Django test runner completed with OK
Failed	0	No failures indicated by the test runner output
Execution time	26.604 seconds	Latest terminal screenshot
Django system check	Passed	Screenshot: no issues identified (0 silenced)
Integration status	Final verification in progress	Teammate says frontend/backend changes were integrated and merge conflicts resolved; changes remain under final verification
GitHub Actions CI	Latest integrated run not confirmed	Earlier CI success was reported; no new CI run link/result supplied for this integrated version

Result statement: The latest supplied screenshot shows **59 tests run in 26.604 seconds**, followed by **OK** and test database cleanup. It also shows the Django system check identified no issues (0 silenced). The teammate reports that this is the integrated frontend/backend test suite, up from 27 tests on the earlier frontend branch. The integrated changes are still undergoing final verification.

These results apply to the test run shown and do not by themselves prove full system acceptance, performance, security, complete SRS coverage or a successful CI run for the latest integrated changes.

8.3 CI/CD

SCRUM-11 is assigned to Shivansh Sharma. The goal is for GitHub Actions to run configured checks and automated tests on pushes and pull requests. The teammate reports the CI checks passed. Retain a workflow run URL or screenshot in

Jira/GitHub as evidence and verify the workflow is triggered by expected events.

8.4 Evidence to retain

Keep the three test files committed; retain the terminal screenshot/output in SCRUM-10; link the successful GitHub Actions run in SCRUM-11 or the README; add coverage/defect metrics only if actually measured.

9. Sprint 1 Plan and Reconstructed Record

9.1 Proposed Sprint Goal

Establish the initial foundation for the Project Management System by analyzing requirements, agreeing the high-level Django architecture and database design, preparing the repository and application foundations, beginning the project-management feature, and preparing test and CI/CD foundations.

9.2 Sprint backlog and task ownership

Jira item	Task	Assignee shown
SCRUM-3	Analyze Project Requirements	Shivanagouda I Patil
SCRUM-4	Design System Architecture	Shibi R Shetty
SCRUM-5	Design Database Schema	Shibi R Shetty
SCRUM-6	Set Up GitHub Repository	Shivansh Sharma
SCRUM-7	Implement Project Creation and Management	Shibi R Shetty
SCRUM-8	Develop Frontend Foundation	Sheetal S Bevinakatti
SCRUM-9	Develop Backend Foundation	Shibi R Shetty
SCRUM-10	Develop Unit & Integration Test Cases	Sheetal S Bevinakatti
SCRUM-11	Set Up CI/CD Pipeline	Shivansh Sharma
SCRUM-12	Project Documentation	Shivanagouda I Patil

9.3 Decisions and items to verify

• Technology direction: the team stated that it agreed to use Python + Django. The architecture proposes Django Templates, HTML/CSS/JavaScript and PostgreSQL; confirm PostgreSQL as the final database choice.
• Repository: the README documents a GitHub repository and proposed structure. Shivansh owns repository setup in the shown Jira assignment.
• Functional scope: SCRUM-7 covers project creation/management. Confirm whether Sprint 1 includes create/view/edit only or also archive/delete and membership management.
• Sprint dates: the screenshot shows 1 Oct–8 Oct. Confirm these dates and update Jira if needed.
• Authentication, task management, Kanban/workflow and other features remain in the broader SRS, but the shown backlog does not assign each a separate implementation task.

9.4 Action items

1. Confirm Sprint Goal, dates and backlog scope in Jira.
2. Confirm database choice and align README, architecture, schema and Django configuration.
3. Define acceptance criteria for each Jira item before marking it Done.
4. Develop unit/integration tests alongside implementation and record real results.
5. Verify GitHub Actions using an actual push or pull request.
6. Keep the SRS, architecture, database design, README and status documentation synchronized.

9.5 Progress status

The supplied materials confirm that Jira tasks were created and assigned and that design/documentation files exist. They do not prove every assigned task has been implemented or accepted. Update statuses using Jira and actual repository/test evidence; do not treat this reconstructed note as evidence that the sprint was completed.

10. Documentation Inventory and Maintenance

Document/artifact	Purpose	Maintenance rule
SRS.pdf	Approved product requirements baseline	Change through team review; retain version history.
Sprint1_Requirements_Analysis.pdf	SCRUM-3 analysis and proposed sprint scope	Update if the team changes selected requirements.
PMS_System_Architecture_Design.pdf	High-level architecture and request flow	Align technology/modules/database with implementation.
SASD_Specification.docx	Detailed architecture/design specification	Update open issues, interfaces and decisions.
Database schema PDF / ERD / SQL	Proposed logical model and PostgreSQL DDL	Validate against approved Django models and migrations.
Software-Test-Plan.pdf/.docx	Testing strategy and planned coverage	Update when scope, tools or schedule changes.
README.md	Repository guide and setup instructions	Verify commands and paths whenever repository changes.
Jira and sprint records	Ownership, status, decisions and blockers	Record actual decisions and progress.
GitHub Actions/test evidence	Automated checks	Use actual run links/logs and defects/resolutions.

10.1 Suggested docs folder contents

Keep approved documents under docs/ with stable filenames. Avoid unnecessary duplicate PDFs. If editable and PDF versions are both retained, identify the current approved copy and keep them consistent.

Suggested structure:

```
docs/
SRS.pdf
Sprint1_Requirements_Analysis.pdf
PMS_System_Architecture_Design.pdf
SASD_Specification.docx
pms_database_schema_design.pdf
pms_database_er_diagram.png
pms_database_schema_postgresql.sql
Software-Test-Plan.pdf
Project_Documentation.pdf
Sprint1_Planning_Record_DRAFT.md
```

11. Open Issues and Risks

Issue/risk	Why it matters	Next action
Database configuration mismatch	README mentions PostgreSQL in production and SQLite locally; architecture/schema emphasize PostgreSQL.	Confirm environments and document actual settings.
Designs are proposed/in progress	Schema and architecture files do not prove implementation.	Compare against Django settings, models and migrations.
Sprint date ambiguity	Jira shows 1 Oct–8 Oct; design files are dated 26 October 2026.	Confirm sprint dates and current status.
No contemporaneous meeting notes	No actual meeting minutes were supplied.	Review this reconstructed record; correct/remove assumptions.
No test execution results supplied	STP is a draft plan, not evidence of passed tests.	Add actual test output and workflow links.
General roles vs Jira assignments differ	README module roles differ from some task allocations.	Keep both views clear and update by team agreement.
Full SRS exceeds Sprint 1 backlog	Many features lack separate implementation tasks in the screenshot.	Agree selected scope and avoid implying all features are complete.

12. SCRUM-12 Completion Checklist

- Consolidated documentation is stored in the repository.
- SRS and SCRUM-3 analysis are linked and consistent with the agreed scope.
- Architecture and database choices are confirmed or explicitly marked proposed.
- README setup instructions match the actual repository and Django configuration.
- Test plan is available; actual test cases/results are linked when they exist.
- Sprint goal, backlog, assignments and action items have been reviewed by the team.
- Open decisions and status gaps are visible, not silently assumed.
- Team has reviewed this document and corrected inaccurate or outdated information.

13. References

1. Project Management System — Software Requirements Specification, Version 1.0, Team 11, September 2026.
2. Software Architecture and Design Specification, Version 1.0, status In Progress, October 2026.
3. System Architecture Design — Project Management System, Version 1.0, 26 October 2026.
4. Project Management System — Database Schema Design, Version 1.0, 26 October 2026, proposed design for team review.
5. Software Test Plan, Version 1.0, 16 September 2026, status Draft.
6. Project repository README.md.
7. Sprint 1 Jira task list and assignment screenshots supplied in the conversation.
8. Sprint 1 Requirements Analysis PDF prepared for SCRUM-3.